

1. Overview

The **Medical RAG Assistant** is an intelligent, retrieval-augmented generation (RAG) system designed to provide safe, context-aware medical answers focused on:

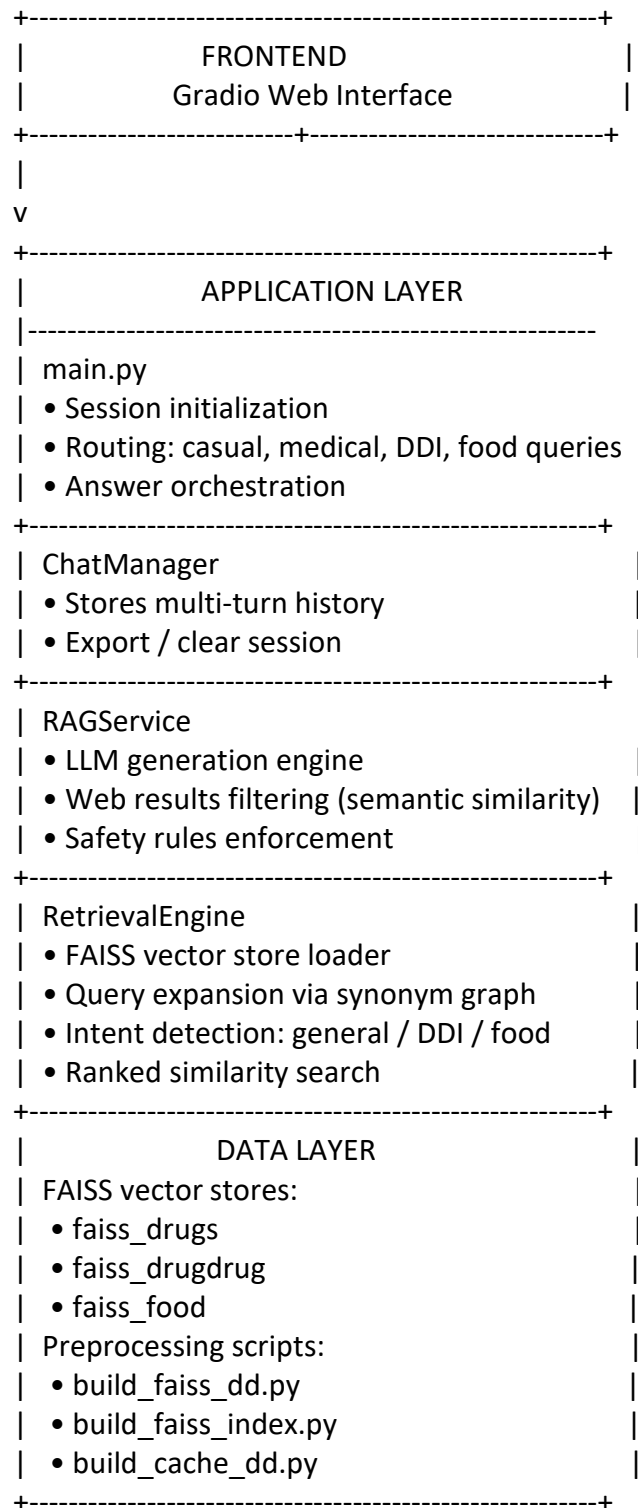
- Drug–drug interactions
- Drug–food interactions
- General drug information
- Safety warnings
- Limited casual conversation (greetings + identity queries)

The system integrates:

- Large Language Models (LLM)**
- Custom FAISS vector search indices**
- Synonym-enhanced retrieval**
- Multi-dataset ingestion (DrugBank-derived CSVs)**
- Google Search (optional)**
- Gradio UI interface**
- Session-based chat memory**

It is designed to follow strict medical-safety generation rules and avoid hallucinations or unsafe recommendations.

2. System Architecture



3. Key Components

3.1 main.py — System Orchestration

Responsibilities:

- Load configuration

- Initialize ChatManager, RAGService, RetrievalEngine

- Provide a unified `answer_question()` function

- Route queries:

Query Type	Handler
Greeting / Identity	Casual generator
Non-medical query	Static rejection
Medical query	Full RAG pipeline

Also prepares:

- Chat reset function

- Export session function

- Launches UI via Gradio

3.2 ChatManager — Session Management

Features:

- Stores conversation per session ID

- Limits history length (trimming older messages)

- Formats history for LLM

- Exports the session to JSON

- Creates human-readable summaries

3.3 RetrievalEngine — Vector Search & Intent Routing

Tasks:

Load all FAISS indices

Build a **synonym map** from drug metadata

Intent detection:

general

ddi

food

Expand queries using synonyms

Remove duplicates

Merge multiple FAISS results

Score normalization → $1 - \text{distance}$

Retrieval flow:

User Query

→ intent detection

→ expansion

→ FAISS search

→ scoring

→ deduplication

→ top-k results

3.4 RAGService — LLM + Web Search Safety Layer

Includes:

Model loading (Qwen-0.5B fine-tuned)

Text generation pipeline

Sentence-Transformer similarity filtering for SERP results

Strict safety + anti-hallucination rules:

Use only provided context

No medical prescriptions

Ask for clarification when ambiguous

Follow-ups MUST use chat history

Must warn users to consult doctors

Two modes:

generate_with_context() — full RAG generation

generate_casual() — greetings only

4. Data Pipeline

4.1 Drug–Drug FAISS Builder (build_faiss_dd.py)

Steps:

Preprocess CSV

Collapse duplicate interactions

Limit max 30 interactions per pair

Create FAISS-ready documents

Embed using MiniLM

Save FAISS index

4.2 Multi-Dataset Builder (build_faiss_index.py)

Creates:

faiss_drugs

faiss_food

Using memory-efficient row generators.

4.3 Metadata Cache Builder (build_cache_dd.py)

Creates all_docs_cache.json containing:

- drug IDs

- names

- interaction summaries

- synonyms

Enables synonym expansion in RetrievalEngine.

5. User Interface (UI)

Implemented in **Gradio (Blocks)**.

Features:

- Real-time chat interface

- Example quick-questions

- Export chat

- Clear chat

- Automatic session ID

- Two-column responsive layout

6. Configuration (config.py)

Centralized config supports:

- Model selection

- Embedding model path

FAISS index paths

Google API keys

Chat history limits

Temperature, max tokens

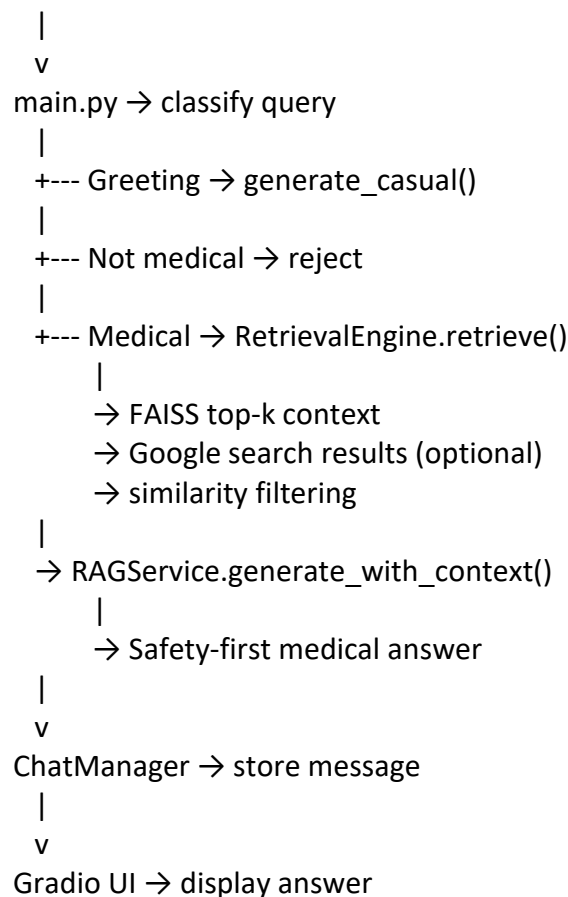
Gradio server settings

Supports loading environment variables via:

`Config.from_env()`

7. RAG Flow (End-to-End)

User Query



8. Safety Mechanisms

The system enforces critical safety rules:

No prescription

No guessing

Use only context

Doctor disclaimers

Detect ambiguous queries

Reject non-medical conversations

Strict memory-bounded history

9. Deployment

Supports:

CPU only

CUDA if available

Custom ports

Offline mode (FAISS only)

Public Gradio sharing

10. File Structure

```
project/
|
|— main.py
|— config.py
|— core/
|   |— chat_manager.py
|   |— rag_service.py
|   |— retrieval_engine.py
|
|— ui/
|   └─ gradio_app.py
|
```



```
└─ build_faiss_dd.py
└─ build_faiss_index.py
└─ build_cache_dd.py
|
└─ faiss_indices/
|   └─ faiss_drugs/
|   └─ faiss_drugdrug/
|   └─ faiss_food/
|
└─ utils/
```

11. Conclusion

This system is a **production-grade medical RAG pipeline** integrating:

- Structured data ingestion
- Semantic search
- Strict safety controls
- Multi-turn conversational reasoning
- Flexible UI layer
- External web augmentation

It is designed for hospitals, research labs, AI health startups, or academic work needing safe, explainable medical interaction analysis.